

FCTCONNECT
(Proyecto de desarrollo)

Ciclo Formativo de Grado Superior (CFGS):
Desarrollo de Aplicaciones Web

Autores:
David Barbosa Olayo, Marc Mancilla Pontejo

Grupo: DAW2
Curso académico: 2024-2025
Centro: IES Puig Castellar

Versión del proyecto: v0.1.0-draft (Memoria inicial)

Licencia:
Esta obra está sujeta a una licencia de Reconocimiento-NoComercial-SinObraDerivada 3.0 España de Creative Commons.

Resumen del proyecto

Cada curso escolar, los centros de Formación Profesional se enfrentan al mismo problema: encontrar, coordinar y hacer seguimiento de prácticas de calidad (FCT) para todos sus alumnos, gestionando al mismo tiempo las expectativas de las empresas colaboradoras y la carga de trabajo de los tutores de centro. En la práctica, este proceso suele apoyarse en hojas de cálculo, correos dispersos y documentación en papel, lo que genera duplicidades, pérdida de información y una enorme inversión de tiempo en tareas administrativas de bajo valor.

FCTConnect nace como una aplicación web destinada a centralizar todo el ciclo de vida de la FCT: desde el alta de empresas y ofertas, hasta la asignación de alumnos, el seguimiento de horas y la evaluación final. El sistema proporcionará un portal único para todos los actores implicados —alumnado, empresas, tutores de centro y coordinadores de FCT— permitiendo registrar datos, gestionar candidaturas, validar asignaciones y generar informes desde una interfaz web moderna, accesible y segura.

En el lado del servidor, un backend desarrollado con Java y Spring Boot expondrá una API REST que gestionará usuarios, roles, ofertas de prácticas, candidaturas y evaluaciones, persistiendo la información en una base de datos relacional (PostgreSQL o MySQL). La capa de seguridad se implementará con Spring Security y JWT, definiendo permisos diferenciados según el rol (ALUMNO, EMPRESA, TUTOR_CENTRO, COORDINADOR, ADMIN). En el lado del cliente, un frontend en Angular ofrecerá una experiencia de usuario responsive, con paneles específicos para cada rol, formularios validados y vistas de seguimiento en tiempo real del progreso de las prácticas.

El proyecto se plantea como un desarrollo full-stack end-to-end: desde el modelado del dominio educativo (ciclos, grupos, alumnos, empresas, convenios, seguimientos) hasta la

planificación de pruebas, despliegue y futura extensibilidad. Más allá de digitalizar formularios, FCTConnect aspira a mejorar la transparencia del proceso, facilitar la búsqueda de prácticas “decentes y buenas” para los alumnos y reducir la carga burocrática del profesorado y del centro.

Palabras clave

Formación en Centros de Trabajo

Gestión de prácticas

Aplicación web

Spring Boot

Angular

REST API

JWT

PostgreSQL

Asignación de alumnos

Seguimiento de FCT

Digitalización de procesos

Centro educativo

Abstract

Every academic year, vocational training centers face the same challenge: finding, coordinating and monitoring high-quality internships (FCT) for all their students, while managing the expectations of partner companies and the workload of school tutors. In practice, this process is often based on spreadsheets, scattered emails and paper documents, which leads to duplicated information, data loss and a huge amount of time spent on low-value administrative tasks.

FCTConnect is a web application designed to centralize the entire FCT lifecycle: from company and offer registration to student assignment, hour tracking and final evaluation. The system will provide a single portal for all stakeholders —students, companies, school tutors and FCT coordinators— allowing them to register data, manage applications, validate assignments and generate reports from a modern, accessible and secure web interface.

On the server side, a backend built with Java and Spring Boot will expose a REST API that manages users, roles, internship offers, applications and assessments, storing data in a relational database (PostgreSQL or MySQL). The security layer will be implemented with Spring Security and JWT, defining different permissions according to role (STUDENT, COMPANY, SCHOOL_TUTOR, COORDINATOR, ADMIN). On the client side, an Angular-based frontend will offer a responsive user experience with dedicated dashboards for each role, validated forms and real-time views of internship progress.

This project is conceived as an end-to-end full-stack development: from modeling the educational domain (programs, groups, students, companies, agreements, follow-ups) to planning tests, deployment and future extensibility. Beyond simply digitizing forms,

FCTConnect aims to improve process transparency, help students access good-quality internships and reduce the administrative workload of teachers and educational centers.

Keywords

Internship Management
Vocational Training
Web Application
Spring Boot
Angular
REST API
JWT
PostgreSQL
Student Assignment
FCT Tracking
Process Digitalization
Educational Center

Índice

Portada
Resumen del proyecto
Abstract
Índice

1. Introducción
 - 1.1 Contexto
 - 1.2 Justificación
 - 1.3 Objetivos
 - 1.3.1 Objetivo general
 - 1.3.2 Objetivos específicos
 - 1.4 Estrategia y Planificación del Proyecto
 - 1.4.1 Fases del desarrollo
 - 1.5 Metodología de Trabajo
 - 1.6 Estudio Económico y Presupuestario
2. Descripción del Proyecto
 - 2.1 Análisis de Requisitos
 - 2.1.1 Requisitos funcionales
 - 2.1.2 Requisitos no funcionales
 - 2.2 Tecnologías
 - 2.2.1 Comparativa de las tecnologías valoradas
 - 2.2.2 Tecnologías escogidas
 - 2.3 Estructura del Proyecto
 - 2.4 Descripción de los Componentes

2.5 Definición de las Funcionalidades

3. Pruebas y Validación
 - 3.1 Estrategia de pruebas
 - 3.2 Pruebas funcionales manuales
 - 3.3 Consideraciones sobre integración y automatización
 - 3.4 Pruebas de rendimiento
 4. Conclusiones
 - 4.1 Conclusiones generales del proyecto
 - 4.2 Consecución de los objetivos
 - 4.3 Valoración de la metodología y planificación
 - 4.3.1 Aspectos positivos
 - 4.3.2 Áreas de mejora
 - 4.4 Visión de futuro
 - 4.5 Conclusión final
 5. Glosario
 6. Bibliografía
 7. Anexos
-

1. Introducción

En la Formación Profesional actual, la Formación en Centros de Trabajo (FCT) se ha convertido en una pieza clave para la inserción laboral del alumnado. No se trata únicamente de “cumplir horas” en una empresa, sino de asegurar prácticas de calidad, ajustadas al perfil del ciclo formativo y a las expectativas del estudiante y del entorno productivo. Sin embargo, el proceso de gestión de las FCT sigue siendo, en muchos centros, intensivo en papel, hojas de cálculo y correos electrónicos dispersos.

Cada año se repite el mismo patrón: los coordinadores de FCT recopilan datos de alumnos, buscan empresas colaboradoras, negocian plazas, gestionan documentación legal, asignan alumnos a puestos, controlan horarios y siguen la evolución de las prácticas a través de informes, llamadas o visitas puntuales. Este flujo suele implicar recopilar información en múltiples formatos, reenviar correos, revisar versiones de documentos y mantener un “puzzle” de datos que reside en diferentes aplicaciones y personas. El resultado es un proceso frágil, con riesgo de errores, duplicidades y falta de transparencia para los propios alumnos.

FCTConnect se plantea como respuesta a este problema: una aplicación web que centralice la gestión de las FCT, desde la perspectiva del centro educativo y de los distintos actores que intervienen. No se trata solo de registrar datos, sino de ofrecer un entorno donde:

- Los alumnos puedan conocer las ofertas disponibles, expresar preferencias, adjuntar su CV y seguir el estado de sus candidaturas.
- Las empresas puedan registrar ofertas claras, gestionar plazas, validar horas y comunicar incidencias.
- Los tutores de centro y coordinadores de FCT dispongan de una visión global del proceso, con paneles de control, alertas y herramientas de seguimiento.

El proyecto se aborda desde una perspectiva full-stack: un backend robusto con Java y Spring Boot, una base de datos relacional que modele adecuadamente el dominio FCT y un frontend en Angular que proporcione una experiencia web moderna y adaptada a dispositivos móviles. Esta memoria inicial presenta el contexto del problema, la justificación del proyecto, los objetivos planteados y la planificación de su desarrollo, así como una primera definición de requisitos y tecnologías.

Más allá de la solución puntual, FCTConnect pretende explorar cómo la ingeniería de software puede mejorar procesos educativos reales, reduciendo carga administrativa, aumentando la trazabilidad de la información y contribuyendo a que las prácticas del alumnado sean más justas, transparentes y alineadas con sus intereses profesionales.

1.1 Contexto

La Formación en Centros de Trabajo es un módulo obligatorio en la mayoría de ciclos formativos de FP, entre ellos el CFGS de Desarrollo de Aplicaciones Web. La normativa establece un número determinado de horas, la necesidad de firmar convenios y anexos, y la figura de un tutor de centro y un tutor de empresa que acompañan al alumno durante su estancia.

En muchos centros, la gestión actual se apoya en:

- Hojas de cálculo para listar alumnos, empresas, plazas y asignaciones.
- Formularios en papel para convenios, anexos y registros de seguimiento.
- Correos electrónicos o llamadas para coordinar horarios e incidencias.
- Carpetas físicas o archivos dispersos en unidades compartidas.

Este enfoque presenta varios problemas:

- **Falta de centralización:** la información se fragmenta entre diferentes documentos y aplicaciones.
- **Riesgo de errores y desactualización:** diferentes versiones del mismo listado circulan entre coordinadores, tutores y jefatura de estudios.

- **Dificultad para encontrar “buenas prácticas”:** la valoración de las empresas y plazas suele estar en la cabeza del profesorado, pero no se registra de forma estructurada.
- **Poca visibilidad para el alumnado:** los estudiantes a menudo desconocen qué empresas colaboran habitualmente, qué ofrecen y qué feedback han recibido de antiguos alumnos.

Además, el aumento del número de ciclos, grupos y alumnos hace que la carga administrativa de la FCT sea cada vez mayor. En este contexto, disponer de una herramienta específica para la gestión de FCT permite:

- Normalizar el proceso de alta de empresas, ofertas y asignaciones.
- Registrar de manera estructurada las preferencias y el perfil del alumnado.
- Hacer seguimiento de la FCT mediante informes periódicos y estados actualizados.
- Generar documentación e informes de forma más ágil y con menos errores.

FCTConnect se inserta en este contexto como una solución diseñada específicamente para centros de FP, con foco en el flujo real de trabajo de coordinadores, tutores, alumnado y empresas.

1.2 Justificación

La justificación del proyecto se apoya en tres ejes principales:

1. Problema real en los centros educativos

La gestión de FCT es un proceso crítico y recurrente. Un error en la asignación de prácticas, un incumplimiento de horarios o un problema con la documentación puede afectar directamente al alumno y a la imagen del centro. A día de hoy, muchos de estos procesos se apoyan en herramientas genéricas (Excel, correo, carpetas compartidas) que no están pensadas para este uso concreto.

2. Necesidad de mejorar la calidad y transparencia de las prácticas

Conseguir “prácticas decentes y buenas” no es solo una cuestión de encontrar empresas, sino de poder evaluar la experiencia de los alumnos, identificar empresas que ofrecen buenas condiciones y detectar aquellas que no cumplen los estándares esperados. Sin una herramienta que agregue y estructure estos datos, es difícil tomar decisiones informadas curso tras curso.

3. Oportunidad formativa y tecnológica

Desde el punto de vista del ciclo de DAW, desarrollar FCTConnect supone trabajar con un stack tecnológico muy cercano al entorno profesional: Java + Spring Boot en el backend, Angular en el frontend, una base de datos relacional y una arquitectura

REST. El proyecto permite aplicar conceptos de diseño de bases de datos, seguridad, autenticación basada en tokens, diseño de API, diseño de interfaces y pruebas.

FCTConnect pretende, por tanto, responder a una necesidad real del contexto educativo mientras sirve como proyecto integrador de las competencias del ciclo. La aplicación no busca ser un “gestor genérico”, sino un sistema alineado con el flujo de trabajo de la FCT, con roles, pantallas y procesos diseñados específicamente para este ámbito.

1.3 Objetivos

1.3.1 Objetivo general

Desarrollar una aplicación web para la gestión integral de la Formación en Centros de Trabajo (FCT) que centralice la información de alumnos, empresas, ofertas, asignaciones y seguimientos, facilitando la coordinación entre todas las partes implicadas y mejorando la calidad y transparencia de las prácticas.

1.3.2 Objetivos específicos

1. Modelado del dominio FCT

- Diseñar un modelo de datos que represente alumnos, empresas, ciclos, ofertas de prácticas, candidaturas, convenios, seguimientos e informes.
- Incluir atributos que permitan valorar la calidad de las empresas (historial, valoraciones, incidencias).

2. Gestión de usuarios y roles

- Implementar un sistema de autenticación y autorización basado en Spring Security y JWT.
- Definir roles diferenciados (ALUMNO, EMPRESA, TUTOR_CENTRO, COORDINADOR, ADMIN) con permisos específicos.

3. Gestión de empresas y ofertas de FCT

- Permitir que el centro y/o las empresas registren empresas colaboradoras, contactos y datos básicos.
- Registrar ofertas de FCT (descripción del puesto, tecnologías, horario, número de plazas, requisitos).

4. Gestión de alumnos y preferencias

- Registrar datos básicos de alumnos, su ciclo y grupo, así como sus preferencias (áreas de interés, ubicación, horario, transporte, etc.).
- Permitir que los alumnos suban su CV y completen un pequeño perfil profesional.

5. Asignación de alumnos a prácticas

- Implementar un flujo de candidaturas donde el alumno pueda solicitar plazas y el tutor/coordinador pueda aceptar, rechazar o reasignar.
- Proporcionar listados y filtros que faciliten al coordinador encontrar “matches” entre alumnos y ofertas según criterios básicos (tecnologías, disponibilidad, distancia aproximada, etc.).

6. Seguimiento y evaluación

- Permitir registrar seguimientos periódicos (visitas, incidencias, comentarios) por parte del tutor de centro.
- Registrar la evaluación final de la FCT, tanto desde el centro como desde la empresa.

7. Generación y gestión de documentación

- Diseñar la base para generar o adjuntar documentación clave de FCT (convenios, anexos, informes), ya sea mediante plantillas PDF o subida de archivos.

8. Frontend usable y responsive

- Desarrollar un frontend en Angular con módulos diferenciados para cada rol.
- Implementar formularios validados, listados con filtros y paneles de control que faciliten la gestión diaria.

9. Calidad del software

- Definir y aplicar una estrategia de pruebas (unitarias, de integración y funcionales manuales).
- Documentar la API (OpenAPI/Swagger) y el uso de la aplicación (manual de usuario).

Con estos objetivos, el proyecto aspira a ser una solución funcional y extensible, que pueda adaptarse a distintos centros y ciclos formativos con cambios mínimos.

1.4 Estrategia y Planificación del Proyecto

El proyecto se plantea como un desarrollo full-stack que abarca desde el análisis inicial hasta el despliegue de un prototipo funcional. La estrategia es iterativa: cada fase entrega un bloque de funcionalidad que se valida y refina antes de avanzar.

En lugar de apoyarse en herramientas genéricas sin estructura, se parte de un modelado explícito del dominio FCT y de una arquitectura basada en capas: base de datos relacional, backend REST en Spring Boot y frontend en Angular. La idea es que cada iteración deje un incremento utilizable: primero un backend mínimo, luego un frontend que consuma esos endpoints, después la seguridad, y finalmente funcionalidades avanzadas como seguimiento y documentos.

1.4.1 Fases del desarrollo

Las fases propuestas (con tiempos orientativos) son:

Fase 0: Preparación del entorno (1 semana)

- Configuración del repositorio Git (rama main, rama develop y posible rama features).
- Creación del proyecto backend con Spring Initializr (Maven, Java 17/21, Spring Web, Spring Data JPA, Spring Security).
- Creación del proyecto frontend con Angular CLI.
- Definición de convenciones de commits, ramas y estilo de código.
- Configuración de una base de datos local (PostgreSQL/MySQL) y pruebas de conexión.
- Elaboración de primeros diagramas de casos de uso y modelo de datos conceptual.

Fase 1: Modelado de dominio y base de datos (1–2 semanas)

- Diseño del modelo entidad-relación (alumnos, empresas, ofertas, candidaturas, convenios, seguimientos, evaluaciones).
- Traducción del modelo a entidades JPA en el backend.
- Configuración de migraciones de base de datos con Flyway o Liquibase.
- Validación temprana de consultas y relaciones (por ejemplo, un alumno puede tener varias candidaturas, pero una única FCT activa).

Fase 2: Backend core (2 semanas)

- Implementación de la API REST básica para:
 - Gestión de usuarios y roles.
 - CRUD de empresas y ofertas de FCT.
 - CRUD de alumnos y sus preferencias.
- Implementación de autenticación con JWT (login, registro de empresas y creación de usuarios gestionada por el centro).
- Configuración de CORS para permitir el consumo desde el frontend Angular.

Fase 3: Frontend Angular básico (2–3 semanas)

- Configuración de módulos principales (auth, alumnos, empresas, ofertas, administración).
- Implementación de pantallas: login, registro/alta de empresas (si se permite), listado de ofertas, detalle de oferta, perfil de alumno.
- Servicios Angular para consumir la API REST.
- Guards de rutas basados en el rol y almacenamiento del JWT (localStorage/SessionStorage).

Fase 4: Funcionalidades avanzadas FCT (2–3 semanas)

- Flujo de candidaturas (el alumno se postula, el tutor/coordinador acepta/rechaza).
- Paneles de seguimiento para el tutor de centro (estado de FCT, incidencias, horas acumuladas).
- Registro de seguimientos periódicos y comentarios de empresa y tutor.
- Base para generación/gestión de documentos (subida de PDFs, plantillas, etc.).
- Posible módulo de notificaciones por correo (Spring Mail) para cambios importantes (aceptación de candidaturas, incidencias).

Fase 5: Pruebas, ajustes y documentación (1–2 semanas)

- Definición y ejecución de pruebas funcionales manuales con roles reales.
- Implementación de pruebas unitarias básicas en backend (JUnit, Mockito) y pruebas de componentes en Angular.
- Documentación de la API con OpenAPI/Swagger.
- Redacción de la memoria final, manual de usuario y anexos.
- Despliegue de un prototipo (por ejemplo, backend en un servicio cloud y frontend en un hosting estático).

Esta planificación es flexible y se podrá ajustar según avances reales, problemas imprevistos o nuevas necesidades detectadas durante el desarrollo.

1.5 Metodología de Trabajo

Para este proyecto se adoptará una metodología ágil inspirada en Scrum, adaptada a la escala del TFG:

- Iteraciones cortas (sprints) de 1–2 semanas, con objetivos concretos (por ejemplo, “API básica de empresas y ofertas” o “módulo de candidaturas en Angular”).
- Prioridad a tener siempre un prototipo funcional, aunque sea con pocas funcionalidades, frente a desarrollar muchas partes a medias.
- Uso de un tablero Kanban (GitHub Projects, Trello o similar) para organizar tareas en columnas (Por hacer / En progreso / En revisión / Hecho).
- Repositorio Git con ramas de características y pull requests, incluso trabajando en solitario, para fomentar buenas prácticas de control de versiones.

Herramientas previstas:

- **GitHub**: control de versiones, issues, pull requests.
- **Diagrama de Gantt** (Excel o herramienta online): para tener una visión global de las fases y fechas estimadas.
- **Herramientas de modelado** (draw.io / diagrams.net): diagramas de casos de uso, ER, arquitectura.
- **Angular CLI e IntelliJ/VS Code**: como entornos de desarrollo principales.

La idea es favorecer la iteración continua, la revisión frecuente y la adaptación rápida a cambios (por ejemplo, si el tutor sugiere un flujo distinto para la asignación de alumnos).

1.6 Estudio Económico y Presupuestario

Al tratarse de un proyecto académico, los costes económicos directos son muy reducidos, ya que se emplean tecnologías y herramientas de código abierto o con licencias gratuitas para uso educativo:

Costes directos (bajo/nulo)

- Mano de obra: tiempo de análisis, diseño, desarrollo, pruebas y documentación, realizado por el alumno (y, en su caso, por el equipo).
- Infraestructura de desarrollo:
 - IDEs gratuitos (IntelliJ Community, VS Code).
 - GitHub como plataforma de repositorios.
 - Base de datos local (PostgreSQL/MySQL).

Costes a medio/largo plazo (opcional)

- Alojamiento del backend y de la base de datos en la nube (por ejemplo, servicios con plan gratuito o de bajo coste).
- Mantenimiento evolutivo: nuevas funcionalidades, cambios en la normativa de FCT, adaptación a otros ciclos o centros.

En una hipotética implantación real en un centro educativo, el coste principal recaería en:

- Tiempo de adaptación del sistema al contexto concreto del centro.
- Formación inicial de profesorado y personal administrativo.
- Posible contratación de alojamiento en la nube con ciertas garantías de disponibilidad y copia de seguridad.

En términos de coste-beneficio, una herramienta de este tipo puede ahorrar tiempo administrativo cada curso, reducir errores, mejorar el seguimiento de las prácticas y aportar datos objetivos para tomar decisiones sobre la colaboración con empresas.

2. Descripción del Proyecto

En este apartado se detalla cómo se materializará FCTConnect desde el punto de vista técnico: requisitos, tecnologías, arquitectura y componentes principales.

2.1 Análisis de Requisitos

2.1.1 Requisitos funcionales

A nivel funcional, se han identificado los siguientes requisitos principales:

Gestión de usuarios y roles

- RF1. Alta, edición y baja lógica de usuarios (alumnos, empresas, tutores de centro, coordinadores, administradores).
- RF2. Autenticación mediante usuario/contraseña y emisión de tokens JWT.
- RF3. Asignación de rol(es) a cada usuario y control de acceso a funcionalidades según dicho rol.

Gestión de empresas y ofertas de FCT

- RF4. Registro de empresas colaboradoras con datos de contacto, ubicación y sectores.
- RF5. Registro y edición de ofertas de FCT: título del puesto, descripción, número de plazas, requisitos técnicos y personales, horarios, modalidad (presencial, híbrida, remota), observaciones.
- RF6. Posibilidad de marcar ofertas como activas/inactivas y asociarlas a uno o varios ciclos formativos.

Gestión de alumnos y preferencias

- RF7. Registro de alumnos con datos básicos (nombre, ciclo, grupo, correo, teléfono).
- RF8. Registro de preferencias de FCT del alumno (áreas de interés, zona geográfica, horario preferente).
- RF9. Subida de CV u otros documentos relevantes.

Candidaturas y asignación de FCT

- RF10. Permitir que un alumno se postule a una o varias ofertas publicadas.

- RF11. Permitir que el tutor/coordinador valide, rechace o reasigne candidaturas.
- RF12. Visualizar para cada alumno el estado de su candidatura (pendiente, aceptada, rechazada, asignado a empresa X).

Seguimiento y evaluación

- RF13. Registro de seguimientos periódicos por parte del tutor de centro: visitas, llamadas, incidencias.
- RF14. Registro de la valoración de la empresa sobre el alumno (rúbrica, comentarios).
- RF15. Registro de la evaluación final de FCT por parte del centro.

Documentación y reporting

- RF16. Subida de documentos asociados a la FCT (convenios, anexos, partes de horas, informes).
- RF17. Generación de listados de alumnos y empresas asignadas para un ciclo/curso.
- RF18. Exportación de datos esenciales a formatos estándar (CSV/PDF) para informes internos.

Usabilidad y soporte

- RF19. Paneles de control diferenciados por rol (alumno, empresa, tutor, coordinador).
- RF20. Búsqueda y filtrado de ofertas, alumnos y empresas por diferentes criterios (ciclo, estado, sector, etc.).

2.1.2 Requisitos no funcionales

Seguridad

- RNF1. Autenticación basada en tokens JWT, con expiración configurable.
- RNF2. Autorización basada en roles, aplicando restricciones en las capas de servicio/controlador.

- RNF3. Cifrado de contraseñas en base de datos usando algoritmos robustos (BCrypt, PBKDF2 o similar).

Rendimiento y escalabilidad

- RNF4. La aplicación debe poder gestionar sin problemas el volumen de usuarios propio de un centro de FP (varios ciclos, varios grupos por curso) y sus empresas asociadas.
- RNF5. Las operaciones más frecuentes (consultar ofertas, candidaturas, listados) deben responder en tiempos aceptables (<1 s en condiciones normales de red).

Usabilidad y accesibilidad

- RNF6. Interfaz web responsive, usable en escritorio y dispositivos móviles.
- RNF7. Formularios con validación clara de errores y mensajes comprensibles para usuarios no técnicos.

Mantenibilidad

- RNF8. Arquitectura en capas (controladores, servicios, repositorios) en el backend.
- RNF9. Organización modular del frontend (módulos por dominio/rol).
- RNF10. Código documentado y uso de buenas prácticas de nomenclatura y estructuración.

Fiabilidad

- RNF11. Gestión adecuada de errores: mensajes controlados en las respuestas de la API y en la interfaz de usuario.
- RNF12. Uso de transacciones en operaciones sensibles (asignación de alumnos, registro de evaluaciones).

2.2 Tecnologías

La elección tecnológica está guiada por tres criterios: alineación con el temario de DAW, cercanía al entorno profesional y apoyo de una comunidad amplia.

2.2.1 Comparativa de las tecnologías valoradas

Backend: Spring Boot vs Node.js/Express vs Laravel

- **Spring Boot (Java):**

- Ventajas: ecosistema maduro, fuerte integración con seguridad (Spring Security), JPA, validación, etc. Muy utilizado en entornos empresariales.
- Inconvenientes: curva de aprendizaje algo más elevada, especialmente en configuración de seguridad.

- **Node.js + Express (JavaScript/TypeScript):**

- Ventajas: rapidez de prototipado, mismo lenguaje en front y back.
- Inconvenientes: se requiere seleccionar librerías de seguridad, ORM, etc. de forma manual, lo que aumenta la complejidad para un proyecto académico de tiempo limitado.

- **Laravel (PHP):**

- Ventajas: framework muy completo, con muchas funcionalidades integradas (auth, ORM, etc.).
- Inconvenientes: menos alineado con la especialización en Java que el alumno ya ha desarrollado en otros proyectos.

Frontend: Angular vs React vs Vue

- **Angular:**

- Ventajas: framework completo, con arquitectura clara basada en módulos, inyección de dependencias y TypeScript. Muy usado en entornos corporativos.
- Inconvenientes: curva de aprendizaje mayor que otras opciones.

- **React:**

- Ventajas: librería flexible y ampliamente adoptada.
- Inconvenientes: requiere decidir e integrar más piezas (router, gestor de estado, etc.), lo que añade decisiones adicionales.

- **Vue:**
 - Ventajas: curva de aprendizaje suave.
 - Inconvenientes: algo menos presente en el temario y en algunos entornos empresariales frente a Angular/React.

Base de datos: PostgreSQL vs MySQL

- **PostgreSQL:**
 - Ventajas: muy robusta, soporte avanzado de SQL, fuerte comunidad, muy utilizada en backend con Spring.
 - Inconvenientes: requiere una configuración inicial ligera, pero asumible.
- **MySQL/MariaDB:**
 - Ventajas: muy extendidas, familiaridad previa en muchos centros.
 - Inconvenientes: algunas diferencias en SQL estándar y características avanzadas frente a PostgreSQL.

Teniendo en cuenta la experiencia previa y los objetivos del proyecto, se opta por Java + Spring Boot en el backend, Angular en el frontend y PostgreSQL como base de datos principal.

2.2.2 Tecnologías escogidas

Backend

- Java 17/21.
- Spring Boot (Web, Data JPA, Security, Validation, posiblemente Mail).
- Spring Security + JWT para autenticación y autorización.
- Spring Data JPA como capa de acceso a datos.
- PostgreSQL como sistema gestor de base de datos.
- Maven como herramienta de construcción y gestión de dependencias.

Frontend

- Angular (versión LTS vigente).
- TypeScript.
- RxJS.
- Angular Router.
- Librería de componentes de UI (Angular Material o PrimeNG).

Desarrollo, pruebas y despliegue

- Git + GitHub para control de versiones.
- Postman/Insomnia para pruebas de la API.
- JUnit y Mockito para pruebas unitarias en el backend.
- Herramientas de pruebas para Angular (Jasmine/Karma, o testing con Jest si se configura).
- Opcionalmente Docker para levantar la base de datos y/o el backend en contenedores durante el desarrollo.

2.3 Estructura del Proyecto

El proyecto se dividirá en dos grandes módulos (backend y frontend), cada uno con su propia estructura interna.

Backend (Java + Spring Boot)

- **controller**: controladores REST (endpoints) para alumnos, empresas, ofertas, candidaturas, seguimiento, etc.
- **service**: lógica de negocio (reglas de asignación, validaciones, integridad de procesos).
- **repository**: interfaces JPA para acceso a base de datos.
- **model/entity**: entidades JPA que representan tablas.
- **security**: configuración de Spring Security, filtros JWT, detalles de usuario.

- **dto**: objetos de transferencia de datos entre backend y frontend.

Frontend (Angular)

- Módulos principales (por ejemplo):
 - **auth**: login, registro (si procede), recuperación de contraseña.
 - **alumnos**: panel de alumno, perfil, candidaturas.
 - **empresas**: panel de empresa, gestión de ofertas.
 - **fct**: asignaciones, seguimiento, listados.
 - **admin**: gestión de usuarios, configuración básica.

Cada módulo incluirá componentes, servicios, rutas y modelos propios.

2.4 Descripción de los Componentes

A alto nivel, los componentes principales serán:

- **Módulo de autenticación y seguridad**
 - Gestión de login/logout, emisión y validación de JWT.
 - Asignación de roles y protección de rutas en backend y frontend.
- **Módulo de alumnos**
 - Registro de datos personales y académicos.
 - Registro de preferencias y subida de CV.
 - Vista de ofertas disponibles y estado de candidaturas.
- **Módulo de empresas y ofertas**
 - Gestión de empresas colaboradoras: alta, edición, baja lógica.
 - Registro y gestión de ofertas de FCT.
- **Módulo de candidaturas y asignación**

- Flujo de candidatura del alumno a ofertas.
- Panel del tutor/coordinador para validar/rechazar y asignar plazas.
- **Módulo de seguimiento y evaluación**
 - Registro de visitas y contactos.
 - Registro de incidencias.
 - Evaluación de la FCT por empresa y centro.
- **Módulo de documentación e informes**
 - Subida/descarga de documentos asociados a la FCT.
 - Generación de listados para uso interno del centro.

2.5 Definición de las Funcionalidades

Algunas funcionalidades clave, vistas desde el flujo de uso, serán:

1. **Inicio de sesión y acceso según rol**

El usuario introduce sus credenciales, el backend verifica la validez y, en caso afirmativo, genera un JWT que se utiliza para acceder a las distintas rutas y pantallas según el rol asignado.
2. **Alta de empresa y creación de ofertas**

Desde el rol adecuado (centro o empresa) se registran empresas y se crean ofertas de FCT, indicando la información necesaria para que un alumno pueda valorar si la oferta encaja con su perfil.
3. **Gestión del perfil de alumno y preferencias**

El alumno complementa sus datos, adjunta CV y marca sus preferencias de FCT (tecnologías, zonas, horarios).
4. **Candidaturas y asignación de prácticas**

El alumno visualiza ofertas abiertas y se postula a aquellas que le interesan. El tutor/coordinador, a partir de las candidaturas, valida y asigna la plaza correspondiente, registrando la FCT como “activa”.
5. **Seguimiento de la FCT**

Durante el periodo de prácticas, el tutor de centro puede registrar seguimientos periódicos y marcar incidencias, y la empresa puede introducir observaciones sobre el desempeño del alumno.

6. Evaluación final y cierre de FCT

Una vez finalizadas las horas de prácticas, el sistema permite registrar la evaluación final e indicar si la FCT está superada o no, generando la información necesaria para la documentación oficial del módulo.

3. Pruebas y Validación (borrador)

Nota: Este apartado se detallará en la memoria final. A nivel de borrador, se definen las líneas generales.

3.1 Estrategia de pruebas

- Pruebas unitarias en el backend (servicios, lógica de negocio).
- Pruebas de integración sobre la API REST (Spring Boot Test).
- Pruebas manuales de la interfaz Angular para verificar los flujos principales por rol.

3.2 Pruebas funcionales manuales

- Casos de prueba para registro de empresas, creación de ofertas, alta de alumnos, candidatura, asignación, seguimiento y evaluación final.

3.3 Consideraciones sobre integración y automatización

- Posible integración de las pruebas en un pipeline de integración continua con GitHub Actions.

3.4 Pruebas de rendimiento

- Pruebas básicas de rendimiento sobre las operaciones más frecuentes (listado de ofertas, consultas de alumnos, etc.) con un volumen de datos similar al de un centro real.
-

4. Conclusiones (borrador)

Nota: Este capítulo se redactará de forma completa una vez finalizado el desarrollo. Aquí se indican únicamente los epígrafes previstos.

4.1 Conclusiones generales del proyecto

4.2 Consecución de los objetivos

4.3 Valoración de la metodología y planificación

4.3.1 Aspectos positivos

4.3.2 Áreas de mejora

4.4 Visión de futuro

Posibles líneas futuras: integración con sistemas académicos del centro (SGA), generación automática de documentos oficiales, ampliación a otros módulos (por ejemplo, gestión de proyectos, bolsa de trabajo, etc.).

4.5 Conclusión final

5. Glosario (borrador)

FCT (Formación en Centros de Trabajo)

Módulo obligatorio de la FP en el que el alumnado realiza prácticas en una empresa.

Candidatura

Solicitud de un alumno para ocupar una de las plazas asociadas a una oferta de FCT.

Oferta de FCT

Propuesta de prácticas por parte de una empresa, con descripción, requisitos y número de plazas.

Spring Boot

Framework Java que simplifica la creación de aplicaciones web y servicios REST.

Angular

Framework de desarrollo frontend basado en TypeScript para construir aplicaciones web de una sola página (SPA).

JWT (JSON Web Token)

Formato compacto de token utilizado para autenticación y autorización, que puede transportarse en cabeceras HTTP.

PostgreSQL

Sistema gestor de bases de datos relacional de código abierto, robusto y orientado a transacciones.

REST API

Interfaz de programación basada en el estilo REST, que expone recursos a través de HTTP (GET, POST, PUT, DELETE).

6. Bibliografía (borrador)

- [1] Spring. "Spring Boot Reference Documentation".
 - [2] Spring. "Spring Security Reference".
 - [3] PostgreSQL Global Development Group. "PostgreSQL Documentation".
 - [4] Angular. "Angular Official Documentation".
 - [5] OWASP. "OWASP Top Ten Web Application Security Risks".
-

7. Anexos (previstos)

- 7.1 Manual de instalación/usuario (FCTConnect)
- 7.2 Diagramas de casos de uso y modelo entidad-relación
- 7.3 Enlace al repositorio del proyecto (GitHub)